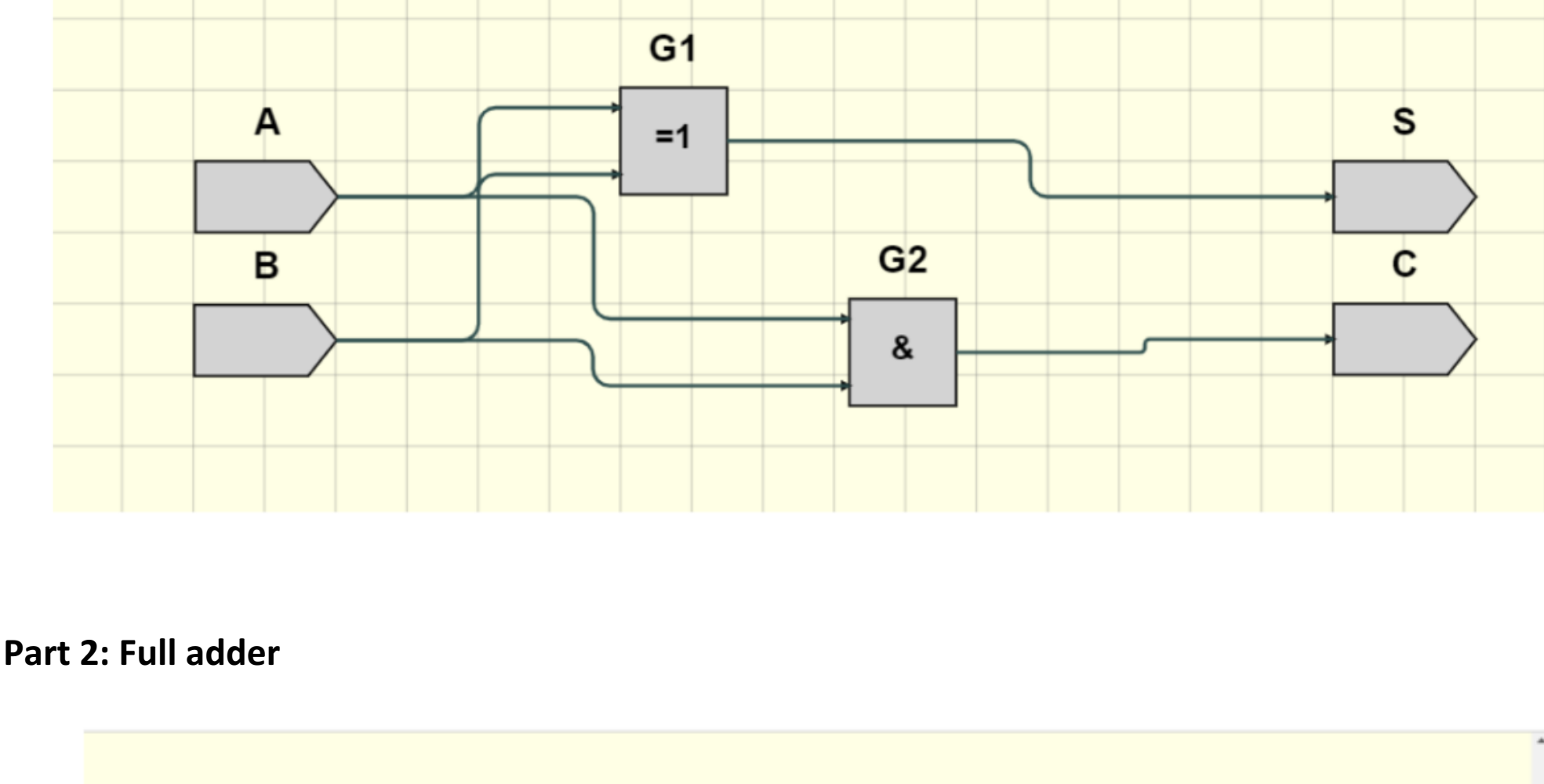
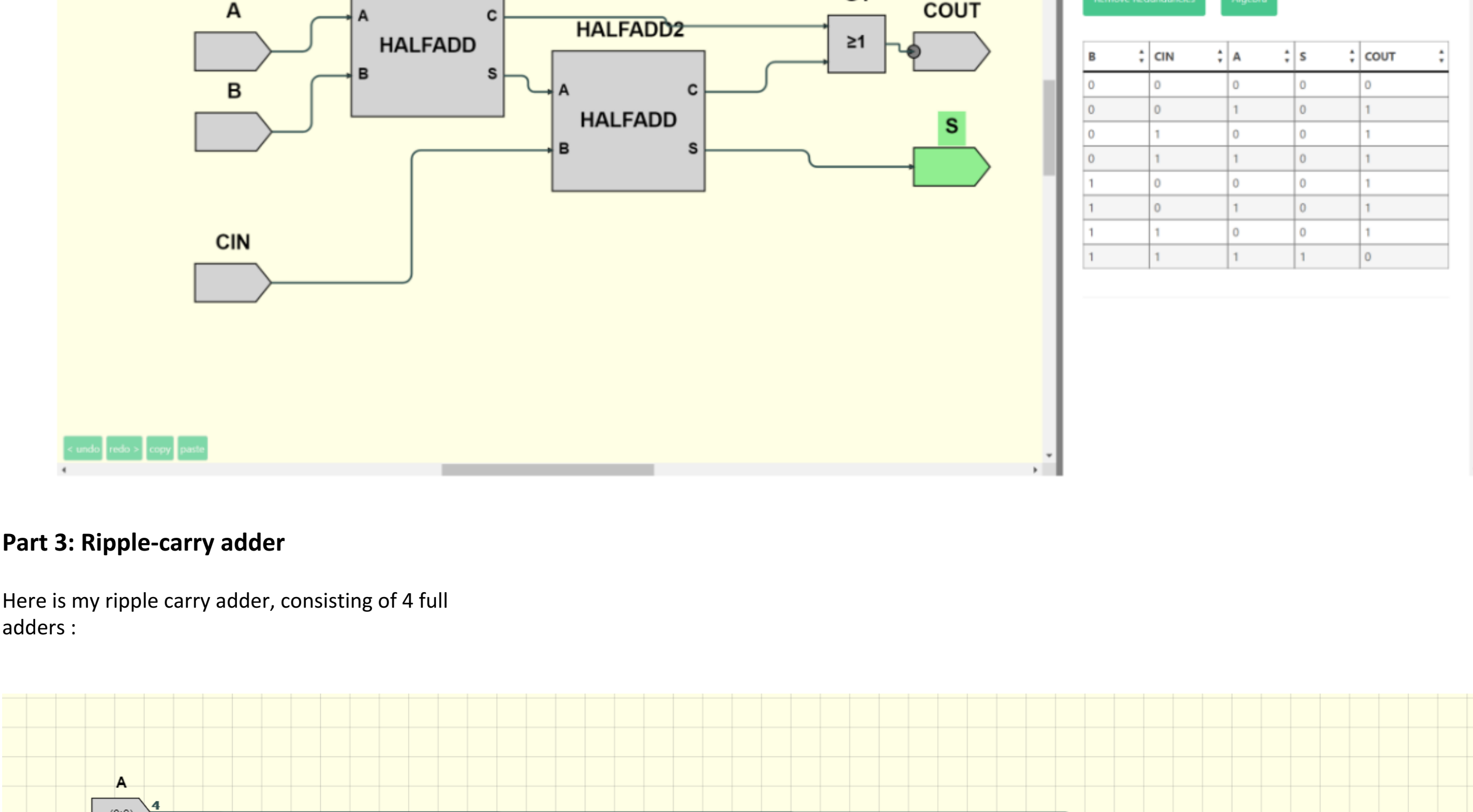


Part 1: Half adder

A half adder, using a XOR and an AND gate:

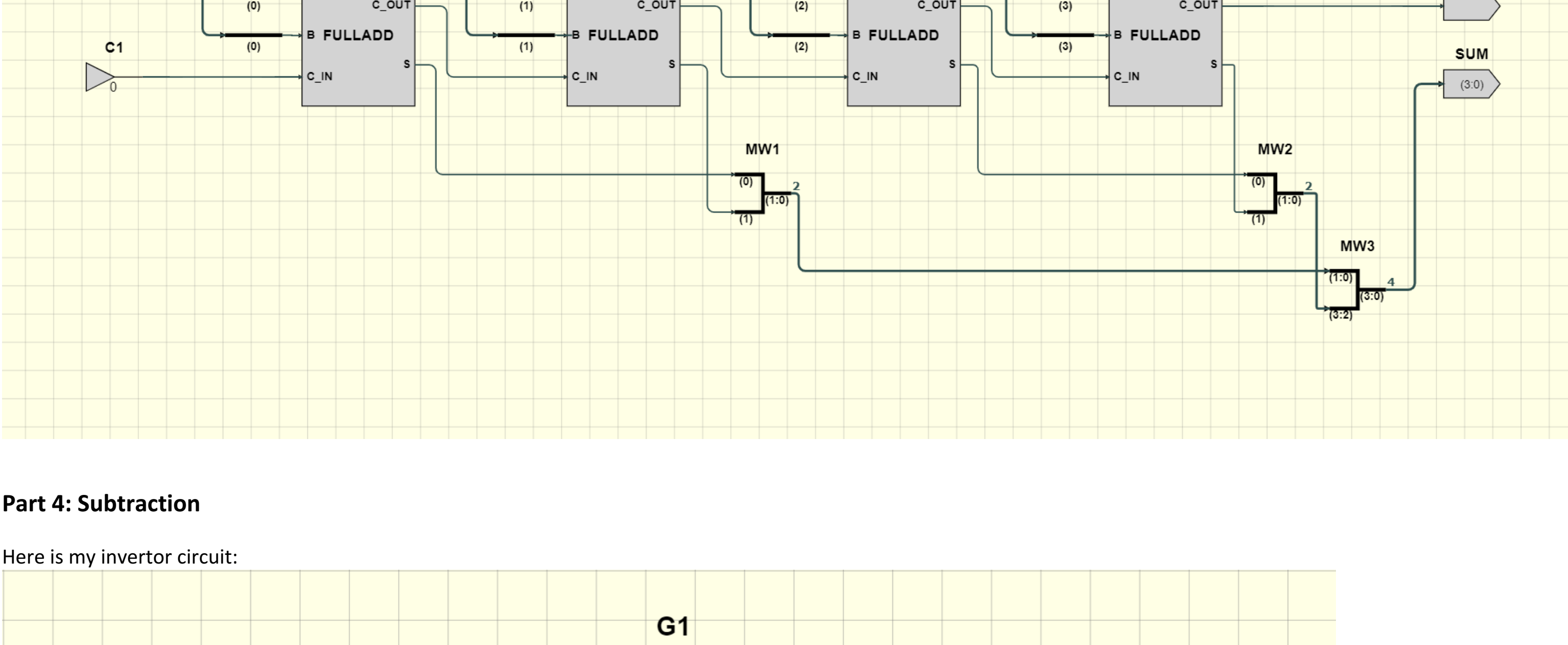


Part 2: Full adder



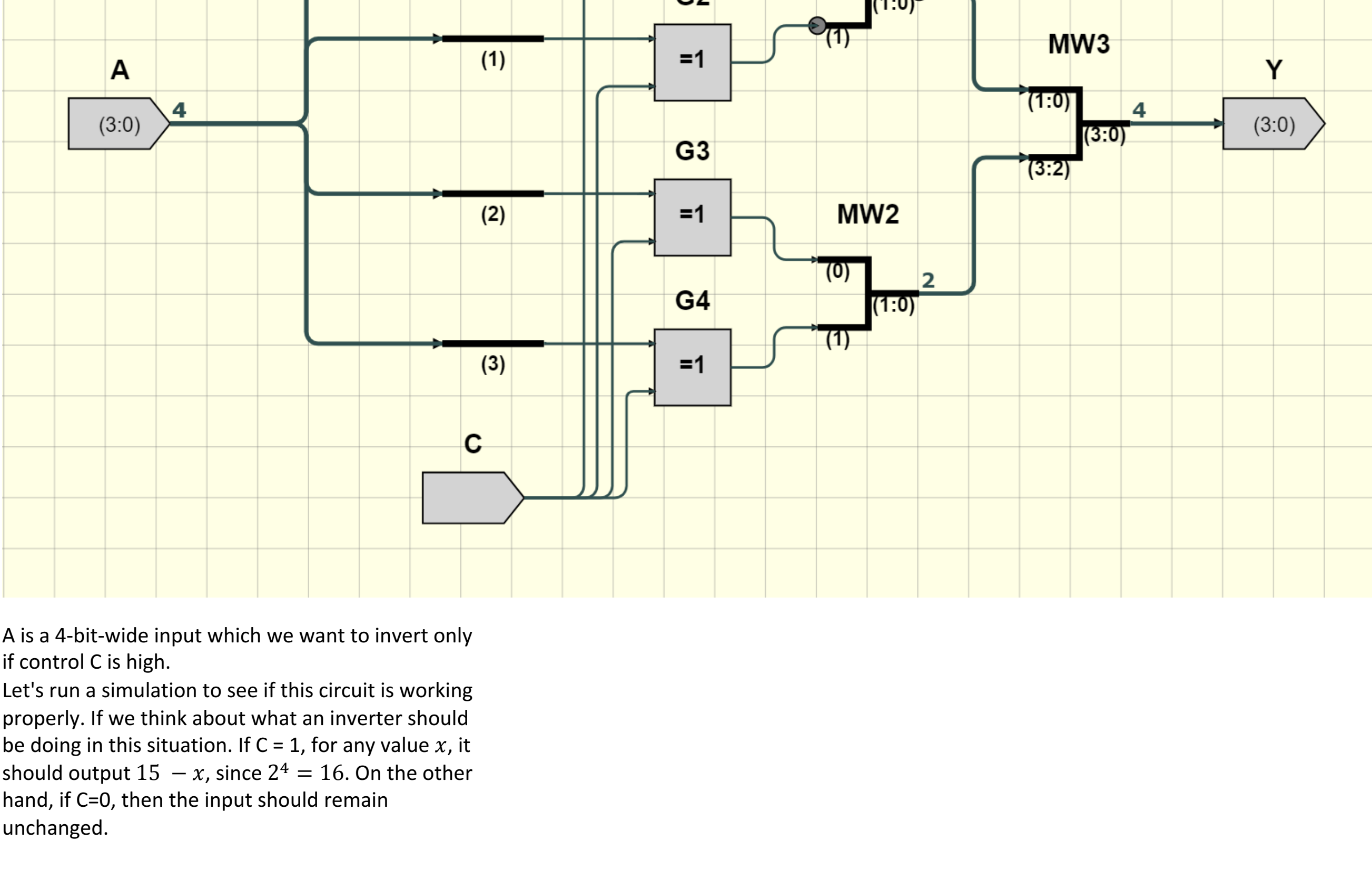
Part 3: Ripple-carry adder

Here is my ripple carry adder, consisting of 4 full adders:



Part 4: Subtraction

Here is my inverter circuit:



A is a 4-bit-wide input which we want to invert only if control C is high. Let's run a simulation to see if this circuit is working properly. If we think about what an inverter should be doing in this situation. If $C = 1$, for any value x , it should output $15 - x$, since $2^4 = 16$. On the other hand, if $C=0$, then the input should remain unchanged.

Let's generate the truth table using Issie and see if this circuit exhibits the expected behaviour.

C	A	Y
1	0	15
1	8	7
1	4	11
1	12	3
1	2	13
1	10	5
1	6	9
1	14	1
1	1	14
1	9	6
1	5	10
1	13	2
1	3	12
1	11	4
1	7	8
1	15	0
0	0	0
0	8	8
0	4	4
0	12	12
0	2	2
0	10	10
0	6	6
0	14	14
0	1	1
0	9	9
0	5	5
0	13	13
0	3	3
0	11	11
0	7	7
0	15	15

In this situation, C is 1, and Y is indeed 15-A. The circuit is working correctly. Let's check the case C=0.

C	A	Y
0	0	0
0	8	8
0	4	4
0	12	12
0	2	2
0	10	10
0	6	6
0	14	14
0	1	1
0	9	9
0	5	5
0	13	13
0	3	3
0	11	11
0	7	7
0	15	15

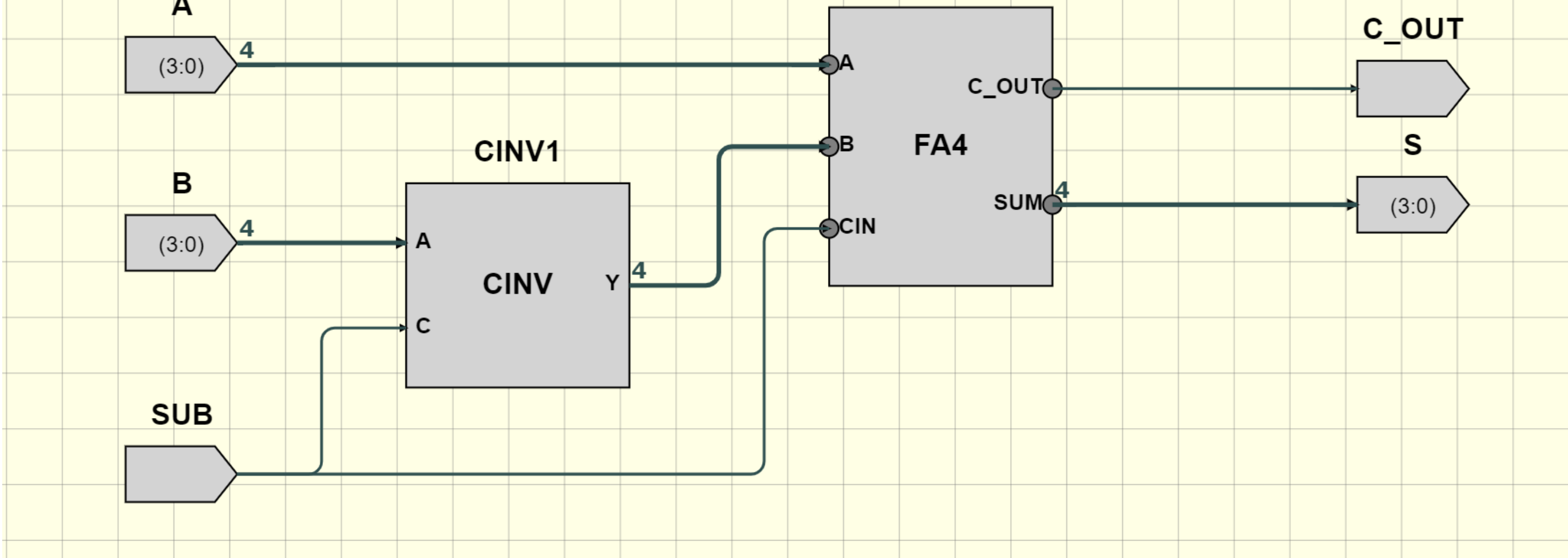
A remains unchanged, which means this is also the expected behaviour. We can therefore move on. We can use this inverter circuit to implement the subtraction operation, using $A + B = A + (-B)$

However, to get $-A$, we need to first invert A, then add 1 (2's complement).

In this case, we use 1 enable bit, which we'll call SUB, both as the enable bit for the inverter circuit built above, and as the bit which will be added to bitwise A to give us $-A$.

Hence we get the following circuit:

(Note I forgot to add the CIN input for fa4, so I need to go fix that before moving forward - I had CIN set to a constant for testing)



Let's generate a truth table to make sure it is working correctly.

B	A	SUB	S	C_OUT
0	0	0	0	0
0	8	0	8	0
0	4	0	4	0
0	12	0	12	0
0	2	0	2	0
0	10	0	10	0
0	6	0	6	0
0	14	0	14	0
0	1	0	1	0
0	9	0	9	0
0	5	0	5	0
0	13	0	13	0
0	3	0	3	0
0	11	0	11	0
0	7	0	7	0
0	15	0	15	0
4	0	0	4	0
4	8	0	12	0
4	4	0	8	0
4	12	0	0	1
4	2	0	6	0
4	10	0	14	0
4	6	0	10	0
4	14	0	2	1
4	1	0	5	0
4	9	0	13	0

This is indeed what we get.

And in the case that SUB is High, then we should be doing subtraction instead of addition:

15	2	1	3	0
15	4	1	5	0
15	8	1	9	0
15	0	1	1	0
7	15	1	8	1
7	7	1	0	1
7	11	1	4	1
7	3	1	12	0
7	13	1	6	1
7	5	1	14	0
7	9	1	2	1
7	1	1	10	0
7	14	1	7	1
7	6	1	15	0
7	10	1	3	1
7	2	1	11	0
7	12	1	5	1
7	4	1	13	0
7	8	1	1	1
7	0	1	9	0
11	15	1	4	1
11	7	1	12	0
11	11	1	0	1
11	3	1	8	0
11	13	1	2	1
11	5	1	10	0

This case is a little harder to interpret, so I simply tested the circuit manually with different values, and it worked correctly.

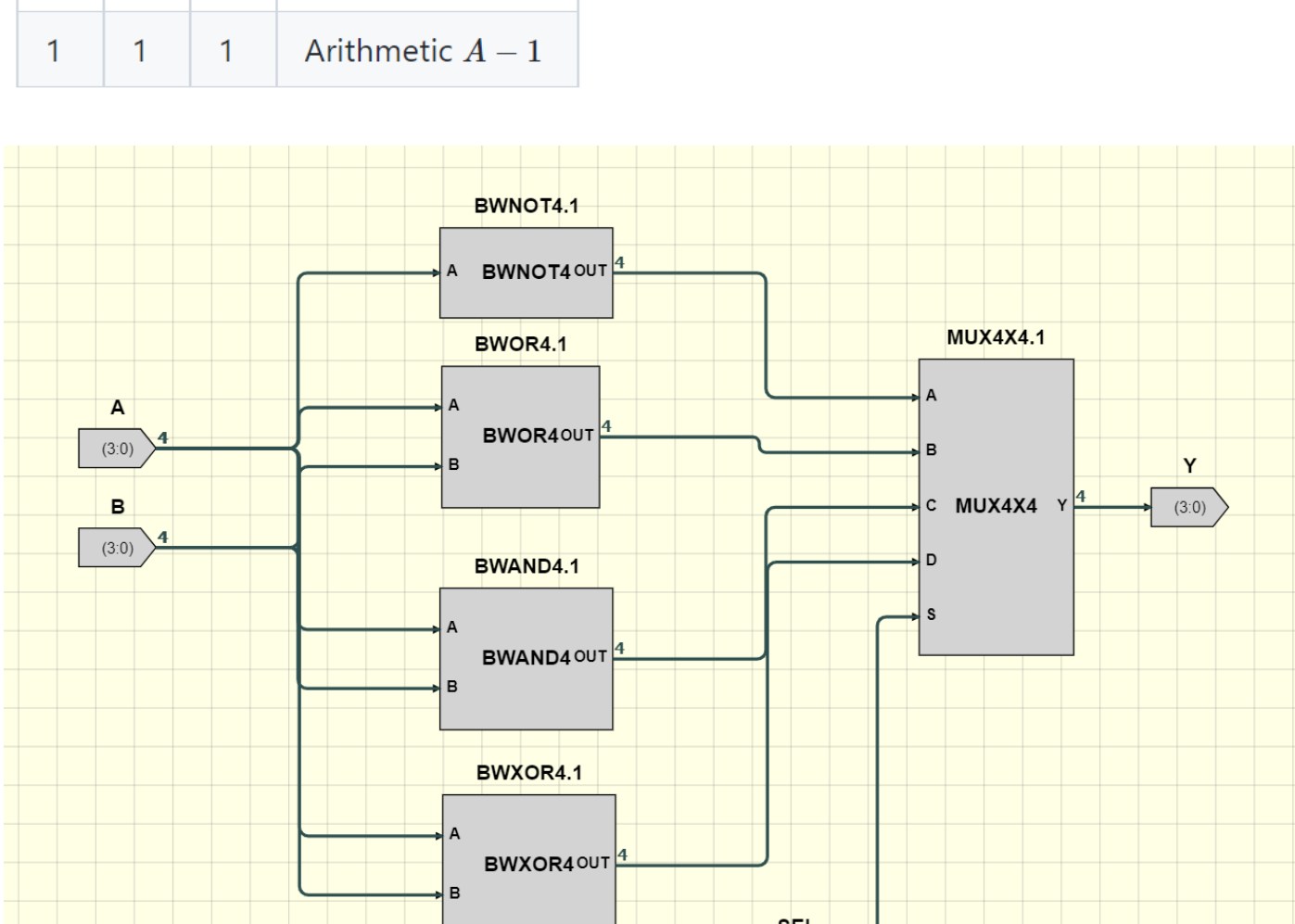
We can move on to the challenge.

Part 5: Challenge - ALU with addition and subtraction

In order to make the arithmetic logic unit with addition and subtraction, we need to implement smaller circuits which will be able to do bitwise operations, just like in the challenge for Part 1 Section 1. Additionally, we will also need our "addsub" block to implement the addition/subtraction property.

Since we have more possible operations this time, our select line needs to be 3 bits instead of 2.

S ₂	S ₁	S ₀	Y
0	0	0	Bitwise $A \wedge B$
0	0	1	Bitwise $A \vee B$
0	1	0	Bitwise $A \oplus B$
0	1	1	Bitwise $A \oplus B$
1	0	0	Arithmetic $A + B$
1	0	1	Arithmetic $A + 1$
1	1	0	Arithmetic $A - B$
1	1	1	Arithmetic $A - 1$



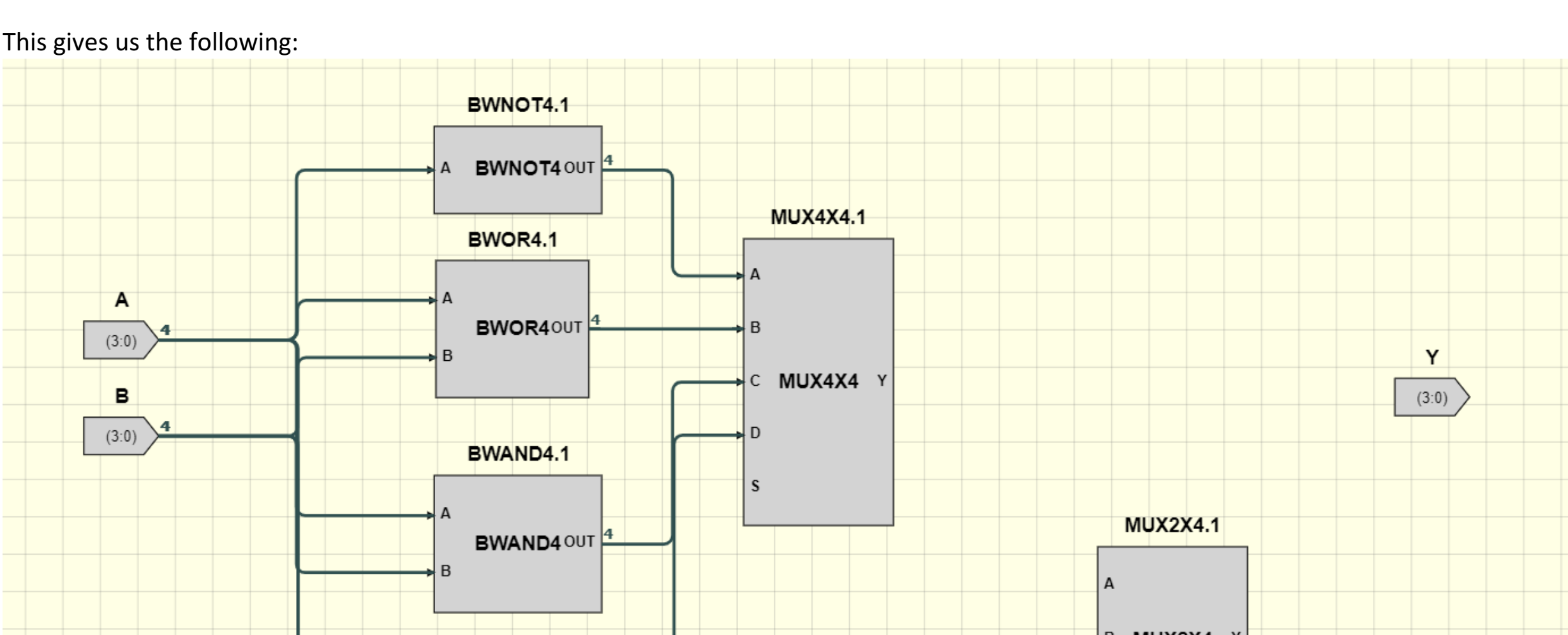
I started with an identical copy of my circuit from the Challenge of Part 1 Section 3. I then deleted the SEL line and created a new one, 3 bits wide this time. I also added the addsub4 block which I made previously. I then added a MUX2x4 block, to switch between the results from the Bitwise operations and the Arithmetic operations.

We can note, from the truth table above, that :

- If $S_2 = 1$, then Y is taking a result from an Arithmetic operation
- If $S_2 = 0$, then Y is taking a result from a Bitwise operation.

Therefore, we can connect the Most Significant Bit of the select line, S₂, to the select line of the 2x4 multiplexer.

This gives us the following:

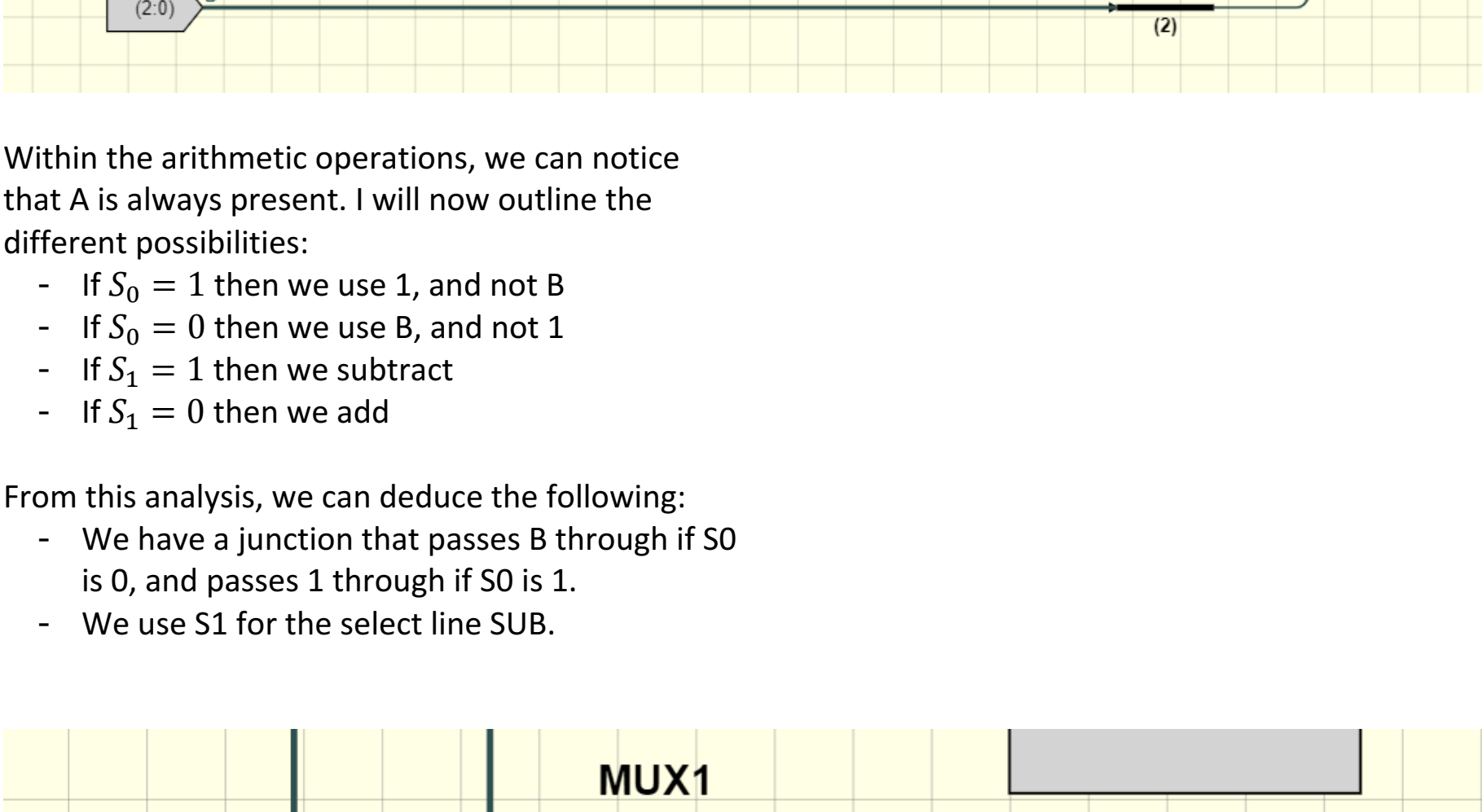


Within the arithmetic operations, we can notice that A is always present. I will now outline the different possibilities:

- If $S_1 = 1$ then we use 1, and not B
- If $S_1 = 0$ then we use B, and not 1
- If $S_1 = 1$ then we subtract
- If $S_1 = 0$ then we add

From this analysis, we can deduce the following:

- We have a junction that passes B through if S₀ is 0, and passes 1 through if S₀ is 1.
- We use S₁ for the select line SUB.



This is what that gives us. Note: the constant 1 must be 4 bits wide (0001) because B is 4 bits wide.

1	0	0	Arithmetic $A + B$
1	0	1	Arithmetic $A + 1$
1	1	0	Arithmetic $A - B$
1	1	1	Arithmetic $A - 1$

We thus get the final design for the ALU:

